Binary Classification of Fake News

Gabriel Vance

DSC 630 – Predictive Analytics

Professor Hua

Milestone 4

Business Understanding

Lies have always been a part of human society and communication. Simply put, lies are saying something that is not true. Fake news is another term for lies. It presents itself as a legitimate update in the world without maintain a factual basis.

When journalism and technology merged, the dispersion of fake news changed from supermarket tabloids and talk radio to webpages, social media, and now AI generated content. It is increasingly more believable, viral, and widespread. Foreign states, activists, and grifters are spreading fake news to achieve their political, influential and financial ends. But while technology is partially responsible for the rise and proliferation of fake news, it can also substantially aid in combating these lies.

For my project, I chose to use a natural language processor to identify fake news. The assumption was that fake news or lies contained a certain language or pattern structure that stood out from truthful news. This is likely due to fake news largely being “spam” or manufactured news, so in essence, machines were generating fake news and left a “watermark” that was easily predictable, like spam or malware.

For data analysis, we will employ traditional Python data analysis tools in conjunction with NLP libraries such as Text Blob and VADER. For predictive modeling, we will utilize libraries like sklearn for generating classification reports and xgboost for training models.

Business Objective: Train a model that predicts fake news with 75% accuracy.

Data Understanding

Collection

The dataset for this notebook was provided by https://www.kaggle.com/datasets/algord/fake-news. I intentionally chose one of the most popular datasets on Kaggle that has already been downloaded over 10,000 times. The dataset contains 626 real news articles and 434 fake news articles. The news articles themselves have been fact-checked by PolitiFact.

It's important to note that this dataset is crucial to understanding how fake news detectors work. The dataset must be vetted and peer reviewed to avoid having a biased set that will influence the model. One bad data point won't throw off the model, but a strong pattern or trend will corrupt the model. Future models might opt for different datasets to verify fake news can be detected on different sources. These news articles were mostly from the 2016 election year.

Data Exploration

Summary of Features

The dataset was comprised of 3 primary features:

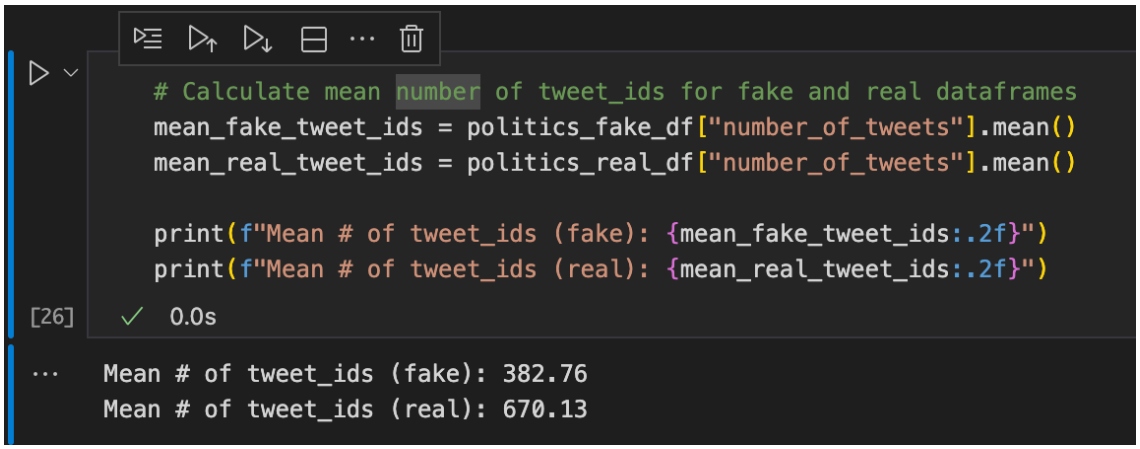
1. News_url – news source URL
2. Title – the title of the article
3. Tweet IDs – The tweets related to the article

Each row is a single instance of an article. The title is the headline of the article. The news URL is the source of the article. Tweet IDs are the number of tweets that shared the article. Other fake news datasets contain the actual text of the article, but this isn't required for classifying fake news. Using only the title allows us to expedite model comparison and training time.

Tweet IDs can be used for manually looking up the actual tweet via the twitter API/URL.

Explore Data (Tweet IDs)

Tweet IDs are essential in capturing the resonance of an article. Real and fake news were tweeted at various rates, but the numbers showed a stark contrast in fake news. Before normalization, the mean tweets were as follows:



Initial impressions were that fake news was tweeted less than real news. But when controlling for 0s, more than a third of all real news had 0 tweets, whereas only 9% of fake news had 0 tweets. Because many articles were tweeted 0 times and we didn't want to risk data loss, I chose np.log1p to standardize the data. The transformation worked as intended, and the

skewness/kurtosis fell close to 0.

```
# Skewness and kurtosis for number_of_tweets
fake_skew_log = skew(politics_fake_df["number_of_tweets_log"])
real_skew_log = skew(politics_real_df["number_of_tweets_log"])
fake_kurt_log = kurtosis(politics_fake_df["number_of_tweets_log"])
real_kurt_log = kurtosis(politics_real_df["number_of_tweets_log"])

print("Before Log Transformation:")

print(f"Fake Articles - Skewness: {fake_skew:.2f}, Kurtosis: {fake_kurt:.2f}")
print(f"Real Articles - Skewness: {real_skew:.2f}, Kurtosis: {real_kurt:.2f}")

print("\nAfter Log Transformation:")

print(f"Fake Articles - Skewness: {fake_skew_log:.2f}, Kurtosis: {fake_kurt_log:.2f}")
print(f"Real Articles - Skewness: {real_skew_log:.2f}, Kurtosis: {real_kurt_log:.2f}")
```

[35] ✓ 0.0s

```
Before Log Transformation:
Fake Articles - Skewness: 13.16, Kurtosis: 196.74
Real Articles - Skewness: 6.55, Kurtosis: 52.10

After Log Transformation:
Fake Articles - Skewness: -0.30, Kurtosis: -0.62
Real Articles - Skewness: 0.61, Kurtosis: -0.95
```

```
# Calculate mean number of tweet_ids for fake and real dataframes
mean_fake_tweet_ids = politics_fake_df["number_of_tweets_log"].mean()
mean_real_tweet_ids = politics_real_df["number_of_tweets_log"].mean()

print(f"Mean # of tweet_ids (fake): {mean_fake_tweet_ids:.2f}")
print(f"Mean # of tweet_ids (real): {mean_real_tweet_ids:.2f}")
```

[34] ✓ 0.0s

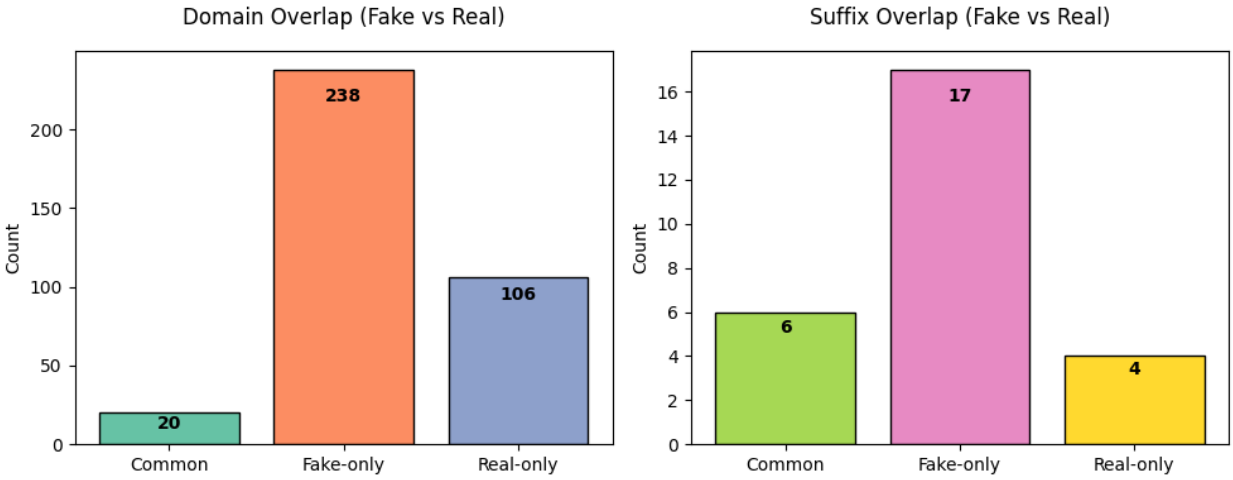
```
Mean # of tweet_ids (fake): 4.04
Mean # of tweet_ids (real): 2.90
```

Likewise, the number of tweets turned in the opposite behavior: fake news is tweeted 38% more than real news. The original, raw data showed higher tweets for real data due to a handful of anomalies pulling the overall mean significantly higher.

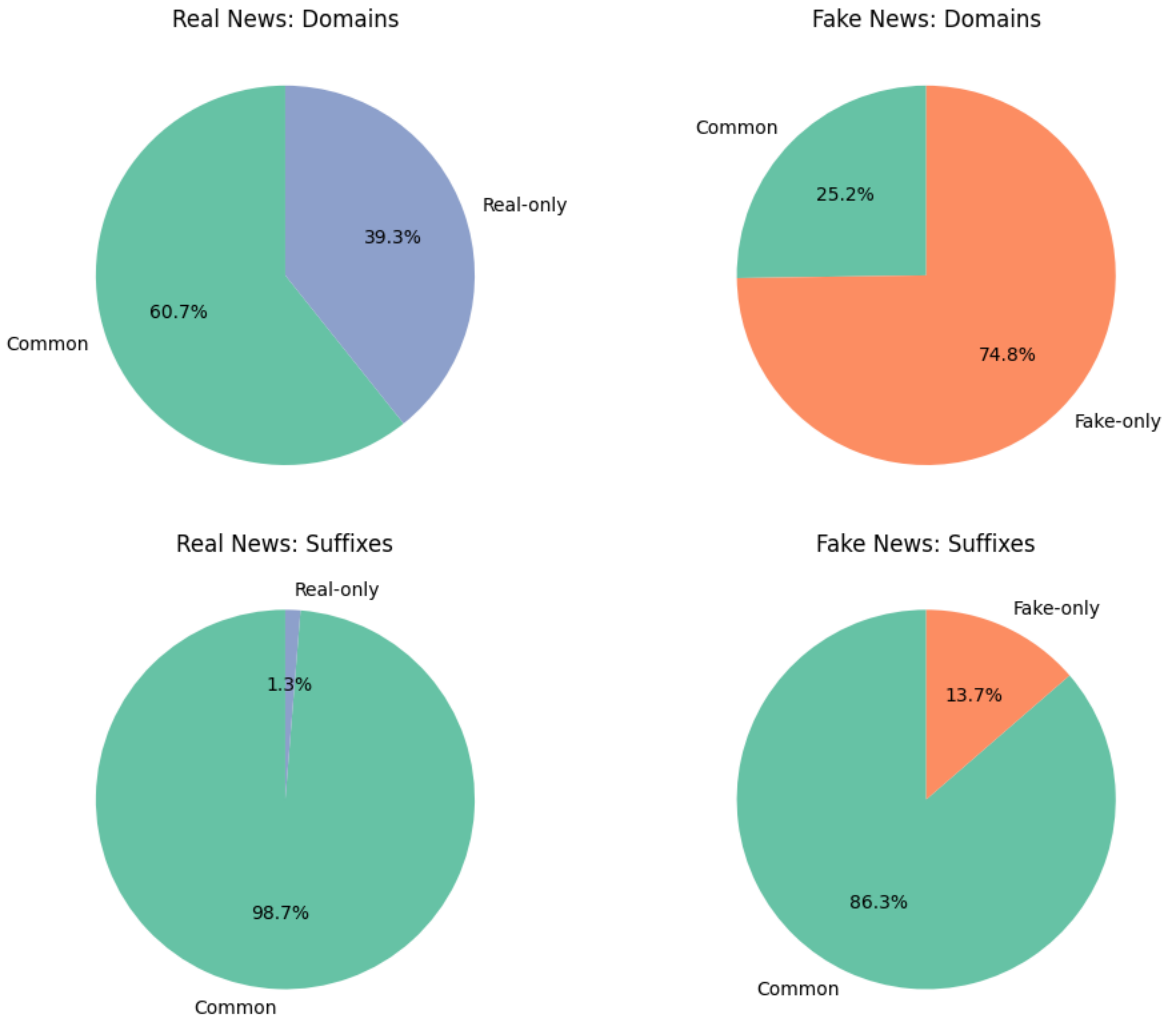
Export Data (Domains)

Domains are the URLs or address that fake news manifests. Much like antivirus software searches for heuristic signatures within malware, so too can we examine the various characteristics of the domains.

For this data, we created comparisons between fake and real data. Common domains existed in both real and fake data frames while “real-only” and “fake-only” domains, a pattern we would continue throughout the project. Comparing domains vs suffixes, fake-only domains appeared to be a key indicator.



But these numbers only capture unique counts. Pie charts show us what percent of news is dominated by common or real/fake-only domains. Again, we see that fake-only domains make up ~75% of all fake news. This was essential in aiding our predictive efforts.

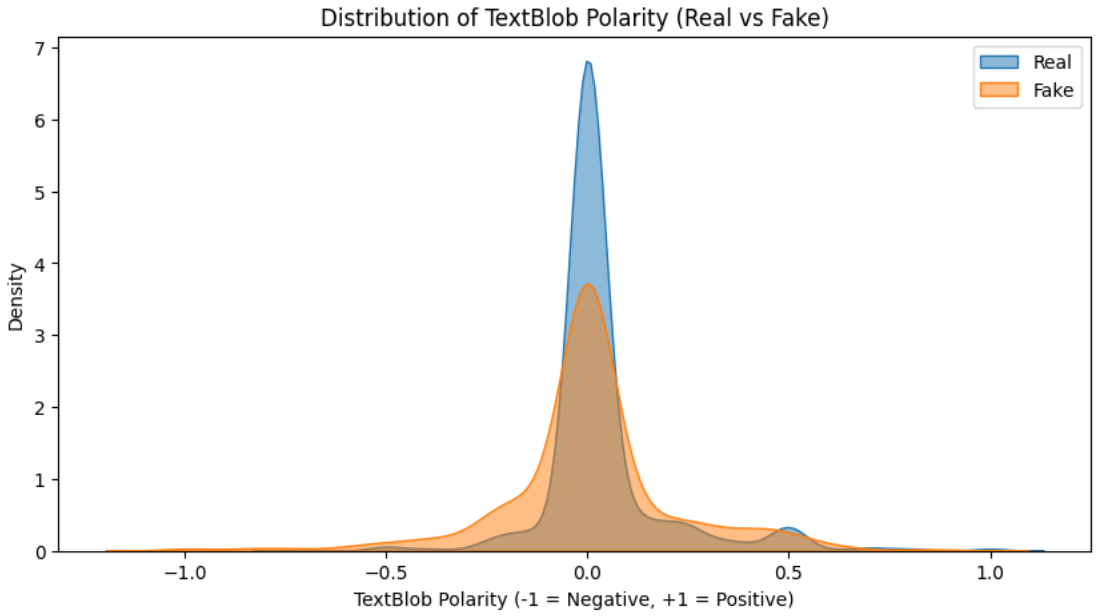


This also provided meaningful data visualizations in the form of word clouds. Word clouds show the frequency of words. When controlling for real-only and fake-only key words, we see various keywords stick out from a visual basis. “whitehouse” and “cbsnews” were high frequency keywords within the real dataset whereas “yournewswise” and “react365” exclusively exist within the fake news.

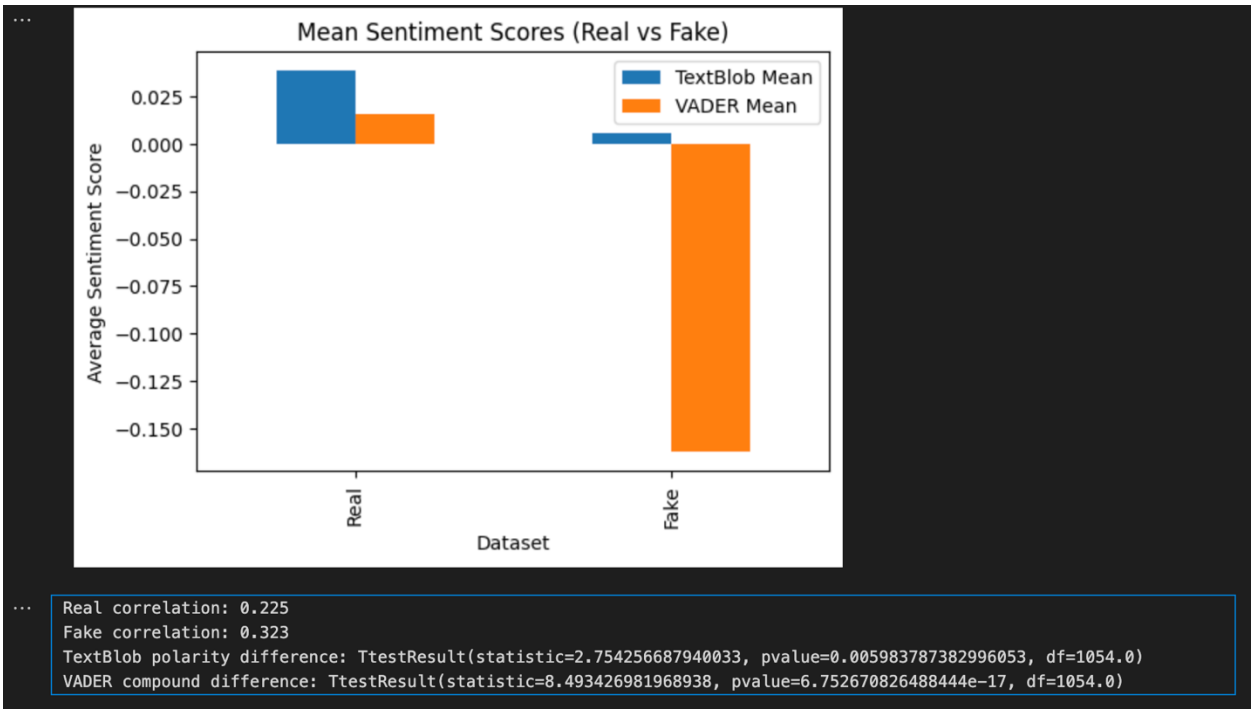


Explore Data (Title)

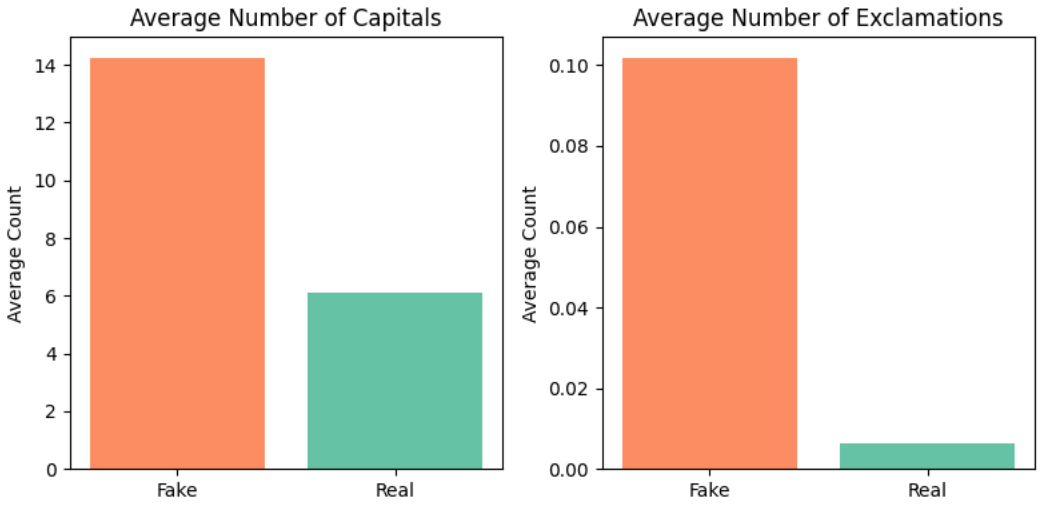
Titles are essential in online journalism and communication. For twitter, titles are the gateway to interest. Inflammatory, polarizing titles drive viewership, or at least our normalized mean tweet counts would seem that way. But now we had to measure just how polarizing and inflammatory these titles were. Utilizing Text Blob and VADER we were able to conduct sentiment values on news. A larger portion of real news maintains a 0.0 polarity, indicating less positive or negative news.



While the sharp contrast is impressive, VADER showed an even larger contrast. So much so that when conducting a t-test, the p-value for VADER sentiment was 6.75e-27 vs Text Blob's 0.006. Both were acceptable for rejecting the null hypothesis, but VADER was more sensitive to the fake news sentiment.



Further driving home, the usage of such polarizing language/writing, we created 2 more features for the number of upper-case characters and the usage of exclamation points. Again, a stark contrast emerges.



Model Evaluation

Model training and evaluation was straight forward. Fake and real data were combined into a single data frame with a “real” label. Fake data had a value of 0 and real data had a value of 1, setting the target for training.

For features, we used the categorical features domain and suffix. Numerical features were Text Blob and VADER sentiment, number of capital letters, number of exclamations, number of words, and number of characters. Title itself was submitted as a Text time, using TfidVectorizer. I fitted a pipeline with a preprocessor that encoded the categorical features.

Using xgboost, our gradient boost classification model ran with 5 folds and param grid, printing the most optimal parameters alongside feature importance. The small number of estimators (100-120) shows that the simple data structure (along with the relatively small sample size) was able to meet our goal of 75% accuracy for both real and fake news. Recall and F1 also maintained values above 75%, indicating that our model is succeeding in retaining high recall and precision. I was a bit surprised to see suffixes so high in the feature importance as the “common” was

concerning. I look forward to loading real data into the model to see its results.

```
Fitting 5 folds for each of 144 candidates, totalling 720 fits
Best Parameters: {'xgb__colsample_bytree': 0.8, 'xgb__learning_rate': 0.2, 'xgb__max_depth': 3, 'xgb__n_estimators': 100, 'xgb__subsample': 1.0}
              precision    recall  f1-score   support

    Fake (0)      0.812     0.793     0.802        87
    Real (1)      0.858     0.872     0.865       125

    accuracy                          0.840       212
   macro avg      0.835     0.833     0.834       212
weighted avg      0.839     0.840     0.839       212


Top 20 features by importance:
       feature  importance
  num_capitals    0.149554
         trump    0.050161
    suffix_com    0.045193
          says    0.033808
vader_compound    0.032718
domain_youtube    0.031701
      breaking    0.026090
    suffix_gov    0.026001
       suffix_    0.025283
    suffix_net    0.023980
          just    0.022335
       senator    0.020686
```

Ethical Considerations

Once the model has shown promise in predicting fake news with high accuracy, a practical use would be to deploy it onto a major social media website such as X or Facebook, where it will detect and block fake news. There are some concerns about freedom of speech, as this model could inadvertently silence someone trying to share an article they found interesting that our model mistakenly views as fake news.

This model is more akin to spam detection. It combines various data points to make a prediction about the validity of the article. While free speech is a tenant of freedom worth respecting and defending, I back reverse searched many of the fake news articles. The domains and the users sharing the fake news often led me to scam websites, often with unsafe unencrypted traffic. The ethical concerns of free speech cannot be tarnished by the evil nature of fake news. These domains are set up to confuse, ensnare, cheat, lie, and polarize readers with lies. Mediums such

as YouTube, X, and Facebook must contend with the reality that their multi-trillion-dollar businesses has to treat this fake news as a harm to their users and platform. This likely means banning the bots reposting the fake news and muting the domains, even if it causes users who believe those domains to quit their platforms.

We must also be considerate of the users reposting fake news. For many of these people, fake news is an identity to their life. They've invested their time (and sometimes money) into believing the lies and bringing them back to the truth is not an easy task. Imagine reading fake news for 10 years only to wake up and discover that your favorite fake news domain has been blocked on Facebook. You would be furious, and that will be the reaction when this model goes live. But failing to implement these changes will result in an internet where people lose their group on reality as they invest more of themselves into lies.